

FOLLOW-UP · 에이전트 셋팅편

작은 모델이 지휘한다

비싼 모델을 기본값에서 끌어내리는 법

라우터 · 웹세션 위임 · 3 줄 설명 · 배치 · 세션 리셋 — 실제 도는 코드로

앞의 두 덩어리는 “요금을 어떻게 읽는가”였다. 이번엔 그 원칙을 매 요청마다 자동으로 적용하는 배선을 짤다.

이 덩어리의 모든 슬라이드는 `agent_setup/router.py` 에 대응 코드가 있습니다. 검증 56/56 PASS.

기본값이 비싸다

대부분의 팀은 "가장 좋은 모델 하나"를 골라 모든 요청에 쓴다. 파일 이름 바꾸기도, 아키텍처 설계도 같은 창구로 간다.

요청	실제 필요한 능력	지금 (기본값)	있어야 할 배선
파일 rename · grep	틀 호출	상위 모델	소형 / 코드
결정된 스펙 구현	코드 생성	상위 모델	중간 모델
방금 짰 코드 설명	설명	상위 모델 · 20 줄	소형 · 3 줄
아키텍처 트레이드오프	추론	상위 모델 · 사고 ON	웹 세션 위임
40 개 사이트 정규화	대량	상위 모델 · 동기	배치 - 50%

요청의 종류를 판정하는 데 드는 비용은 수십 토큰이다. 상위 모델 호출 한 번만 막아도 즉시 회수된다.

당신이 만든 것은 무엇인가

" 이게 라우팅인가 리워크 플로우인가 " — 셋은 층위가 다르다 . 섞어 부르면 처방도 섞인다 .

① 라우팅

요청 1 건을 어느 모델로 보낼지 고른다 .

대상 : 단가 (모델 선택)

효과 : -49~84% (exp06)

단위 : 요청 하나

② 리워크

틀려서 되돌아온 일을 다시 시킨다 .

대상 : 횟수 (턴 반복)

효과 : 48 턴 = 16.6 배 (exp09-C)

단위 : 턴의 누적

③ 워크플로우 컨트롤

파이프라인 전체를 지휘한다 . ①② 를 품는다 .

대상 : 구조 (무엇을 · 어디서 · 몇 번)

효과 : 위 둘의 곱

단위 : 세션 전체

답 : 당신이 설계한 것은 ③ 워크플로우 컨트롤이다 .

라우팅은 그 안의 한 수단이고 , 리워크는 그 컨트롤이 줄이려는 대상이다 . " 리워크 플로우 " 는 되돌아온 작업을 처리하는 흐름을 뜻하므로 목표가 아니라 실패 경로다 .

전체 배선도

작은 모델 (또는 정규식) 이 문 앞에 서서 분류한다 . 그 판정 하나로 이후 비용이 결정된다 .



agent_setup/router.py — route() 한 함수가 위 분기 전부를 담는다 . 의존성 없음 , 오프라인 동작 .

라우터 — 실제 코드

규칙이 먼저다. 정규식으로 잡히면 분류 비용이 0 원이다. 못 잡을 때만 소형 모델을 부른다.

```
RULES = [ # 순서가 곧 우선순위
    (Kind.TOOL,      r" 파일 | 경로 | grep | 검색해 | 커밋 "),
    (Kind.BULK,      r" 일괄 | 정규화 | 라벨링 | \d{2,}( 개 | 건 )"),
    (Kind.REASONING, r" 설계 | 아키텍처 | 트레이드오프 "),
    (Kind.EXPLAIN,   r" 설명 | 무슨 뜻 | 어떻게 동작 "),
    (Kind.IMPLEMENT, r" 구현 | 만들어 | 고쳐 | 리팩터 | 버그 "),
]

def route(task, small_llm=None, *,
         allow_external=True):
    kind, by = classify(task, small_llm)
    if kind is Kind.REASONING and allow_external:
        return Decision(kind, Tier.EXTERNAL,
                        {"mode": "web-chat"})
    if kind is Kind.IMPLEMENT:
        return Decision(kind, Tier.MID,
                        {"explain": "3-lines"})
```

왜 규칙이 먼저인가

분류에 LLM 을 쓰면 그 자체가 요청당 비용이다.

한국어 업무 요청 대다수는 동사 몇 개로 갈린다.

규칙 미스는 소형 모델이 받는다 — 2 단 방어.

판정 정확도

한국어 업무 요청 10 종 테스트 : 10/10 일치.

초기엔 "TODO 전부 grep" 을 BULK 로 오분류 →

TOOL 규칙을 BULK 보다 앞에 두어 해결.

오분류 사례를 남겨두는 이유 — 규칙 기반 라우터는 순서 버그가 대부분이다. 테스트가 그것을 잡는다.

추론은 정액제 창구로 — 그리고 그 대가

긴 추론은 종량 과금에서 가장 비싼 항목이다. 이미 결제 중인 웹 세션에서 끝내고 결론만 주입하면 그 항목이 0 이 된다.

방법 — 로그인된 세션 재사용

- 1. Playwright 로 사용자 프로필 디렉터리를 열어 기존 로그인 쿠키를 그대로 쓴다.
- 2. 문제 정의를 붙여넣고 응답이 끝날 때까지 대기한다.
- 3. 결론을 spec.md 로 저장 → 중간 모델에 입력으로 주입.

효과 : 사고 토큰 과금 0. 앞 덱 실측에서 사고 자동 설정은 240 배였다.

리스크 — 약관을 먼저 읽어라

OpenAI · Google 모두 이용약관에서 자동화된 수단으로의 서비스 접근 · 스크래핑을 제한한다. 계정 정지 사유가 될 수 있다. 업무 데이터를 개인 계정 웹 세션에 넣는 것은 별도의 보안 · 컴플라이언스 문제다. 조직 자산에 이 경로를 넣기 전에 법무 · 보안 확인을 받을 것.

같은 효과를 내는 합법적 대안 — 이쪽을 기본값으로

경로	이점	비용 / 제약	권고
사람이 직접 웹에서 추론	약관 문제 없음	수동 복붙 1 회	권장
API + effort 제한	완전 합법 · 자동화	사고 토큰 일부 과금	권장
Batch API 로 추론 위임	단가 - 50%	지연 최대 24h	조건부
Playwright 자동 조작	종량 0 원	약관 위반 소지 · 정지 위험	비권장

이 슬라이드는 " 하지 말라 " 가 아니라 " 알고 고르라 " 는 뜻이다. 개인 실험과 조직 배포의 기준은 다르다.

구현은 중간 모델로 내려도 되나

스펙이 정해진 구현은 상위 모델을 덜 탄다. 다만 무조건은 아니다.

이렇게 내리면 사고가 난다 (X)

"스펙만 정해졌으면 아무 모델이나 된다"

내려도 되는 기준 (O)

세 조건이 동시에 만족될 때 격차가 좁혀지거나 역전된다

■ ① 스펙이 확정됐을 때

요구사항 분류 과제에서 8B와 상위 모델의 F1 격차는 평균 0.02. 판단이 끝난 일은 크기를 덜 탄다.

■ ② 출력 검증 수단이 있을 때

단위 테스트로 후보를 거를 수 있으면, 고정 컴퓨트 예산에서 7~13B가 34~70B를 최대 15% 상회한다.

■ ③ 임계 크기 이상일 때

3B 미만은 유의하게 떨어진다. 단 Qwen2.5-Coder 1.5B처럼 7B급을 이기는 사례도 있다 — 크기보다 학습 품질.

반대 방향 근거 — pass@1을 10% 올리려면 VRAM 약 4배가 필요하고, Rosetta Code 번역에서는 파라미터 수에 따라 성능이 계속 스케일한다. 스펙이 모호하거나 다단계 추론이 필요하면 격차는 그대로 남는다.

배치 — 단가는 반값, 시간은 약속이 아니다

반값은 사실이다. 문제는 언제 끝나는지를 아무도 약속하지 않는다는 것이다.

벤더	할인	공식 창구	주의
OpenAI Batch	-50%	24 시간 내 완료	미완 시 expired
Google Gemini	-50%	SLO 24 시간	대형 잡은 분할 권장
xAI	-50%	대부분 24 시간 내	best effort
Anthropic	-50%	24 시간 내	대부분 1 시간 내 종료

실제로 깨진 사례

2025년 2월, 기존 1~2 시간에 끝나던 잡이 24 시간에 근접.
 expire 비율이 0% 에서 25% 이상으로 악화된 사례 다수 보고.
 대응: 배치 크기를 절반으로 줄이거나 모델 버전을 고정.

그래서 배치는 세 조건을 모두 만족할 때만 쓴다

- ① 턴 간 의존이 없을 것
에이전트형 다단계 작업은 매 턴 배치 대기가 누적돼 수 시간 ~ 수일이 된다. 부적합.
- ② 마감일 없을 것
자료조사 · 분류 · 정규화 · 임베딩처럼 사람이 기다리지 않는 일.
- ③ **expire 재시도 경로를 설계할 것**
"N 시간 내 미완이면 동기 처리로 전환" 을 코드에 넣는다 — `fallback: sync-after-2h`.

출력을 막는 두 개의 밸브

프롬프트로 " 짧게 답해 " 라고 부탁하는 것은 지켜지지 않는다 . 코드로 잘라야 지켜진다 .

밸브 1 — 설명 3줄 상한

```
def explain_guard(text, max_lines=3):  
    lines = [ 비어있지 않은 줄들 ]  
    if len(lines) <= max_lines: return 그대로  
    return 앞 3 줄 + "(+N 줄 생략 — '자세히' 로 재요청)"
```

구현 직후의 관성적 설명은 소형 모델 + 3 줄 . " 자세히 / 깊게 / 문서로 " 가 있을 때만 full.

밸브 2 — 세션 리셋 트리거

답변을 정규화 (공백 · 문장부호 제거) 해 해시로 기록 .

같은 해시가 2 회 등장 = 메모리에 있는 내용을 다시 생성 중 .

→ 결론을 파일로 커밋하고 새 세션 시작 . 이월 대상은 specs/*.md 와 마지막 결정 사항뿐 .

표기만 바꾼 재설명도 잡힌다 — " 검증합니다 ." vs " 검증합니다 !!" 동일 판정 .

-83%

장황 342tok → 간결 58tok (exp05)

2 회

동일 설명 반복 시 리셋 발동

16.6 배

48 턴 리워크의 비용 (exp09-C)

리셋을 미루면 손해가 복리로 붙는다 — 재생성한 출력이 다시 입력이 되어 이후 모든 턴의 단가를 올린다 .

실패하면 — 모델만 올리지 않는다

라우팅은 첫 배정만 정한다. 돈이 실제로 타는 곳은 실패하고 다시 시작하는 구간이다.

흔한 실패 (X)

같은 범위에 더 비싼 모델을 다시 넣는다.

해야 할 일 (O)

범위와 티어를 같이 올린다. 2 회 이상 실패는 범위가 잘못 잘린 신호다.

시도	티어	다시 여는 범위	이번에 바뀌는 것
1 회차	MID	실패한 함수 / 블록만	첫 배정
2 회차	LARGE	파일 전체 + 호출부	실패 로그 · diff 첨부
3 회차	LARGE	모듈 + 인접 계약	테스트 먼저 작성 강제
4 회차	중단	스펙으로 복귀	모델을 더 올리지 않는다

솔직히 — 이건 절감책이 아니라 보험이다

턴당 단가는 오른다 (2 회차 3.28 배). 대신 4 회차에서 구현을 멈추므로 총비용이 \$0.21 에서 상한이 걸린다. 제자리 재시도는 상한이 없다.

측정 지표는 비용이 아니라 같은 작업이 몇 번 돌아왔는가다 — exp09-C 의 48 턴 리뷰크 (16.6 배) 가 그 경고다.

에스컬레이션은 비싸질까 — 숫자로

" 비싼 모델로 올리면 비용이 폭발하지 않나 " — 정당한 우려다 . 세 가지로 나눠 답한다 .

실패 횟수	제자리 재시도	에스컬레이션	배수
0 회 (성공)	\$0.0272	\$0.0272	동일
1 회	\$0.0545	\$0.1166	2.14 배
2 회	\$0.0818	\$0.2132	2.61 배 ← 최악
3 회	\$0.1090	\$0.2132	1.96 배
10 회	\$0.2998	\$0.2132	0.71 배 ← 역전
48 회 (exp09-C)	\$1.3353	\$0.2132	0.16 배

③ 진짜 비용은 미해결 잔여다 (난제 시나리오 , 4 턴 소진 후)

제자리 재시도 기대 \$0.0804 — 41.0% 확률로 여전히 미해결

에스컬레이션 기대 \$0.1335 — 1.4% 미해결 , 스펙 문서가 남음

개발자 1 시간 \$50 기준 $41\% \times 1h = \$20.5$ 기대손실 . 토큰 5 센트를 아끼려다 \$20 를 태운다 .

① 최악은 2.6 배

실패 2 회 구간이 가장 불리 .

절대값 \$0.08 → \$0.21 (13 센트) .

② 상한이 걸린다

4 회차에 구현을 멈춘다 .

\$0.2132 에서 정지 .

제자리는 선형으로 계속 오른다 .

결론 — 에스컬레이션은 평균 비용을 낮추는 장치가 아니라 꼬리 위험을 자르는 장치다 . 절감은 라우팅이 , 사고 방지는 이쪽이 한다 .

라이브 — 지금 돌려봅니다

python3 agent_setup/test_router.py — 의존성 없음 . 인터넷 없어도 됩니다 .

[1] 분류 정확도

PASS 결제 모듈 아키텍처 설계 트레이드오프 정리해줘 → reasoning
PASS 스펙대로 핸들러 구현해줘 → implement
PASS 방금 짠 코드 설명해줘 → explain
PASS src 밑에서 TODO 전부 grep 해줘 → tool
PASS 경쟁사 40 개 가격 페이지 조사해서 정규화해줘 → bulk

[2] 라우팅 티어 [3] 설명 3 줄 강제

[4] 세션 리셋 트리거 [5] 비용 산식

[6] 리워크 에스컬레이션 [7] 손익분기

기준선 \$0.6500 → 라우팅 \$0.0437 (93.3% 절감)

에스컬 3 턴 \$0.2132 = 제자리 8 턴 \$0.2180

56 건 중 56 PASS / 0 FAIL

시연 순서

1. router.py — 5 개 요청의 행선지
2. 같은 작업을 3 번 실패시켜 에스컬레이션
3. 규칙 하나를 깨뜨려 FAIL 재현

숫자를 읽는 법

93.3% 는 " 전부 상위 모델 " 대비 추정치다 .
실제 절감폭은 요청 구성비에 따라 달라진다 .
PRICES 상수를 각자 단가로 바꿔 다시 재라 .

agent_setup/router.py (라우터) · agent_setup/test_router.py (검증 56 건) — 저장소에 그대로 들어 있습니다 .

내일부터 뭘 커나

전부 한 번에 커지 않는다 . 위험이 낮고 효과가 즉시 보이는 것부터 .

시점	무엇을	어떻게	기대 효과	주의
1 일차	설명 3 줄 상한	프롬프트 + 후처리 컷	즉시 - 40~60% 출력	없음
1 주차	규칙 기반 라우팅	route() 붙이고 로그만 남김	측정 먼저	오분류
2 주차	라우팅 실제 적용	툴 · 설명부터 소형으로	-49~84%	품질 회귀
3 주차	세션 리셋 자동화	반복 감지 → 커밋 → 재시작	컨텍스트 단가 억제	맥락 유실
4 주차	배치 도입	마감 없는 대량 작업만	-50%	expired
보류	웹 세션 자동 조작	법무 확인 전까지 수동	총량 0 원	약관

측정 없이 커지 마라 .

1 주차에 로그만 남기는 이유가 이것이다 . 라우팅 전후의 요청당 비용과 리워크 턴 수를 같이 재야 , 절감이 진짜인지 품질을 팔아 산 것인지 구분된다 .

되돌릴 수 있게 만들어라 — route() 가 항상 상위 모델을 반환하도록 하는 킬 스위치 하나면 롤백이 끝난다 .

정리

- **작은 모델은 답을 만드는 게 아니라 교통을 정리한다**
분류에 드는 수십 토큰이 상위 모델 호출 하나를 막으면 즉시 회수된다.
- **당신이 만든 것은 워크플로우 컨트롤이다**
라우팅은 그 안의 수단이고, 리워크는 그것이 줄이려는 대상이다.
- **구현 티어는 조건을 확인하고 내린다**
스펙 확정 · 검증 수단 · 임계 크기 — 셋이 모일 때만 중간 모델로 내려도 안전하다.
- **배치는 단가를 반으로 줄이지만 시간은 약속하지 않는다**
expired 가 실제로 난다. 폴백 없는 배치는 파이프라인을 멈춘다.
- **밸브는 프롬프트가 아니라 코드로 잠근다**
3 줄 상한과 세션 리셋은 부탁이 아니라 후처리로 강제해야 지켜진다.

세 텍스트를 관통하는 한 문장 — AI가 나쁜 게 아니라 우리가 견적서를 안 읽은 것이다.

Q & A

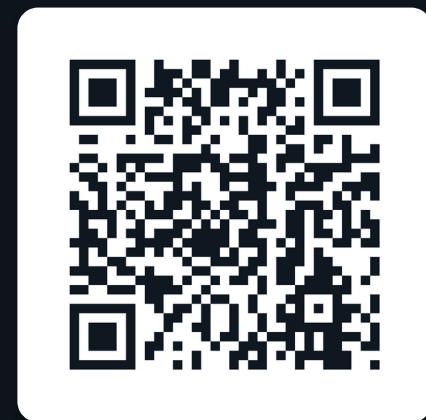
코드는 전부 저장소에 있습니다

agent_setup/router.py — 라우터 · 3 줄 밸브 · 세션 메모리 · 비용 추정

agent_setup/test_router.py — 검증 56 건 (56/56 PASS)

presentation/deckgen/ — 이 덱들을 만든 빌드 스크립트 전부

PRICES 상수만 각자 단가로 바꾸면 자기 팀 숫자가 나옵니다.



github.com/giyeop-cody/token-cost-lab

가져가실 것 하나만 고르라면 — 설명 3 줄 상한 .

오늘 오후에 켤 수 있고 , 되돌리는 데 1 분 걸립니다 .